Version Control Document

Standard Document - Version Control of
Galp Projects #97 & #98

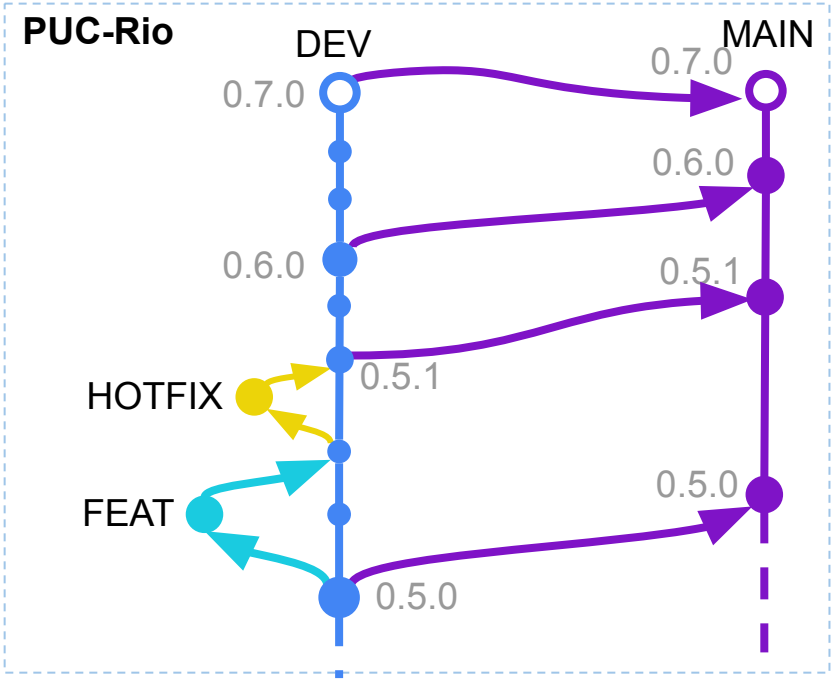
Summary

- Code Versioning
  - Application Environments/Branches
  - Tags and Versions
  - Submodules
  - Environment variables
- Pre-Releases
  - Alpha
  - Beta
  - Gamma
- Pipelines
  - GitHub Actions - PUC-Rio

Code Versioning

Application Environments

- Branch Structure



Code Versioning

Commit name structure

- Commit prefix tags should be written in between brackets
  - Defined tags:
    - **[add]**: new feature inserted (new files, new sections, new code, etc…)
    - **[update]**: change in existing feature
    - **[fix]**: code fixing issue and bugs, or implementing review suggestions
    - **[comment]**: code comments added in existing code
    - **[clean]**: remove non-used code, comments or files
- Commit names composition
  - [add] “feature name” OR “developed component name” OR “new code/comment short description”
  - [update] “feature name” OR “developed component name” : “update short description”
  - [fix] “feature name” OR “developed component name” : “review suggestion” OR “issue or bug name”
  - [comment] “feature name” OR “developed component name” : “comment short description”
  - [clean] “feature name” OR “developed component name”
  - Examples:
    - [add] forgot password
    - [update] combo box in login form: include list options
    - [fix] confirm password in register form: validation checks
    - [comment] route get user: input data structure detail
    - [clean] route new user: deprecated fields deletion

Code Versioning

Branch name structure

- Branches prefix tags should be written before a right bar (/)
  - Defined tags:
    - **feature/**: feature branch, dedicated to add a feature under development
    - **update/**: update a previously developed feature, specs change or evolution.
    - **fix/**: branch to fix an issue or bug in a given feature, or to implementing code review suggestions
    - **doc/**: branch to document code (comments or readme files)
    - **clean/**: branch to remove non-used code, comments or files
    - **test/**: branch to add or update unit or integrated automatic testing capabilities (not merged)
    - **temp/**: temporary branch, dedicated to specific studies or scripts (not merged)

Code Versioning

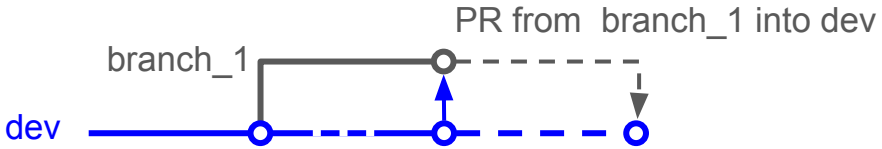
Tags and Versions

- Tags should be placed only on master(production) branch
- Released versions should be tagged according to Semantic Versioning v2.0.0
  - Most important points:
    - A normal version is tagged X.Y.Z where X, Y, and Z are non-negative integers, without leading zeros.
    - X is the major version, Y is the minor version, and Z is the patch/correction version.
      Example: 1.9.0 -> 1.10.0 -> 1.11.0.
    - Major version zero (0.y.z) is for initial development.
    - Major version X (X.y.z | X > 0) increments if any backwards incompatible changes are introduced.
    - Minor version Y (x.Y.z | x > 0) increments if new, backwards compatible functionality is introduced.
    - Patch version Z (x.y.Z | x > 0) increments if only backwards compatible bug fixes are introduced.
    - A pre-release version is denoted by appending a hyphen and an pre-id immediately after the patch version.
      Example1: 1.0.0-alpha ; 1.0.0-beta | Release order example: 1.0.0-alpha.2.0 < 1.0.0-beta.1.5 < 1.0.0
    - **Please read all info in the link above for a more comprehensive understanding.**

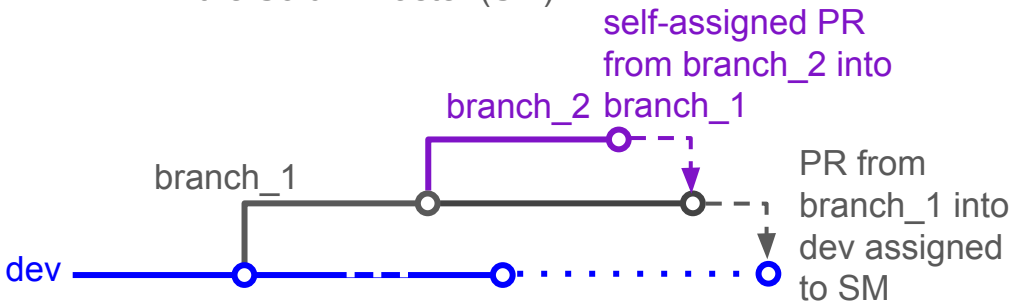
Pipelines

Pull Requests(PR) best practices

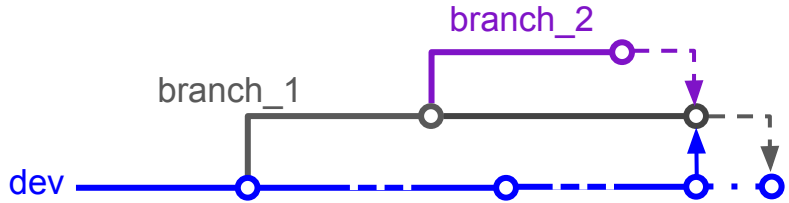
- GitHub - PUC-Rio
  - **Every** branch created from dev, must be tested with the newest state of dev before PR is released.



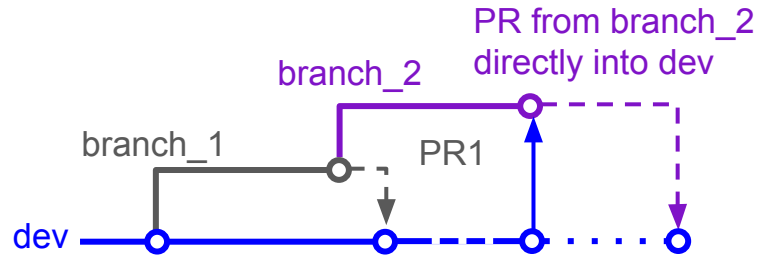
  - **Only** PRs to dev branch must be assigned to the Scrum Master (SM).



  - If a branch is **grandchild** of dev, it should normally be **merged into its parent**, and the **parent merged into** dev.



  - **If and only if** the development of a parent is over and its PR into dev was done, a grandchild branch PR **must** be directly into dev.



Pipelines

CI / CD workflows

- GitHub Actions - PUC-Rio
  - Enforce codebase formatting
  - Check for linting errors
  - Check for typing errors
  - Check for build errors (for deployable libraries/packages)
  - Execute automated tests

Pre-Releases

Project dependent

- 0.1.0a0
  - alpha version is considered the MVP (minimum viable product - Contract specs);
  - Contains the application main structure and it attends the client's main purposes;
  - It usually lacks the final styling, specific features or integration strategies.
- 0.1.0b0
  - beta version presents the lacking specific features (Contract);
  - It presents an enhanced styling and GUI / API optimizations;
  - some integration adjustment must be still needed.

…

- 1.0.0
  - Release with final styling, layout and structured;
  - Full integration with client systems and dependencies.